

Uber Eats Bangalore — Complete Project Guide (Guvi / HCL Data Science)

Use this document to prepare your live evaluation and written explanation. It follows the capstone checklist: domain, objective, ETL, EDA, feature engineering, statistics, SQL, Streamlit, business value.

Part 1 — Domain and problem (what to say first)

Domain (about 3 lines):

Food-delivery platforms like Uber Eats connect customers with restaurants. Success depends on where restaurants operate, how they price meals, which cuisines they offer, customer ratings, and platform features such as online ordering and table booking.

Project in 2 lines:

This project analyzes Bangalore restaurant data from a CSV and order transactions from JSON. The output is a **decision support** tool: a Streamlit app that runs **SQL** and shows results as **tables only** (no charts), similar to internal analytics dashboards.

Objective (2 lines):

Help answer business questions about locations, saturation, pricing segments, cuisines, platform features, and order revenue—so stakeholders can decide onboarding, pricing, and expansion without ad-hoc spreadsheets.

Part 2 — Data sources and files

File	Role
Uber_Eats_data.csv	Restaurant rows: name, location, cuisines, rate, votes, online_order, book_table, cost for two, etc.
orders.json	Array of orders: order_id, restaurant_name, order_date, order_value, discount_used, payment_method
ubereats.db	SQLite file created locally (not required on GitHub; rebuild with create_db.py)

Important: Place CSV and JSON in the **same folder** as the code before running. The GitHub repo typically holds **code only**; you copy data locally.

Part 3 — ETL pipeline (pipeline.py)

Extract:

- Restaurants: `pd.read_csv(...)`.
- Orders: read JSON list into a DataFrame if `orders.json` exists.

Transform (restaurants — clean_restaurants):

1. **Rating:** Remove the text /5, convert to numeric; drop rows where rating is missing after conversion.
2. **Cost:** Remove commas from approx_cost(for two people), convert to numeric.
3. **Rename columns** for SQL-friendly names: e.g. approx_cost_fortwo, restaurant_type, city.

Transform (orders — feature engineering):

Your JSON has **order_date** but the app's SQL groups by **day_of_week** and **month**.

- Parse order_date with Pandas to_datetime.
- Derive day_of_week (e.g. Monday) and month (e.g. Jan).

This is **feature engineering**: creating columns the analysis needs.

If orders.json is missing:

The pipeline generates **synthetic** sample orders (fixed random seed) so the Orders page still works—state this clearly in evaluation if you demo without real JSON.

Load:

DataFrame.to_sql("restaurants", ...) and to_sql("orders", ...) into SQLite.

One-line summary for viva:

"Extract CSV and JSON, clean and rename restaurant fields, derive weekday and month from order dates, load into SQLite for repeatable SQL."

Part 4 — EDA (Exploratory Data Analysis) talking points

Answer briefly using your actual numbers after you run the app:

1. **Size:** How many rows after cleaning restaurants? How many orders?
2. **Types:** Categorical (location, cuisines, online_order) vs numeric (rate, cost, order_value).
3. **Missing values:** How you handled bad ratings (dropped) and cost parsing.
4. **Outliers:** Very high/low order_value or cost—mention if you noticed any.
5. **Distributions:** Typical rating range; low/mid/premium cost segments.
6. **Patterns:** Compare Q3/Q4 (online order and table booking vs average rating).
7. **Cuisine caveat:** cuisines often contains **combined** strings (e.g. "North Indian, Chinese"). Your "most common cuisines" query groups **full strings**, not single cuisine tokens—say this if asked.

Part 5 — Statistical technique (capstone requirement)

Pick **one** test and memorize name + reason:

Example A — Chi-square test of independence

- **Variables:** online_order (Yes/No) vs rating split **high/low** (e.g. median split of rate).
- **Why:** Checks whether two **categorical** variables are associated, without assuming normal distribution.

Example B — Mann–Whitney U

- **Variables:** Rating for online_order = Yes vs No.
- **Why:** Compares two groups when ratings may not be normal.

You can run this in a Jupyter notebook with `scipy.stats`; it does not have to be inside Streamlit.

Part 6 — SQL and business questions (restaurants Q1–Q15)

ID	Idea	SQL ideas you can mention
Q1	Best-rated areas	GROUP BY location, AVG(rate), ORDER BY
Q2	Crowded areas	COUNT(*), GROUP BY location
Q3	Online order vs rating	GROUP BY online_order, AVG(rate)
Q4	Table booking vs rating	GROUP BY book_table
Q5	Which price band has best avg rating	CASE for Low/Mid/Premium, AVG(rate)
Q6	All segments side by side	Same CASE, multiple aggregates
Q7	Frequent cuisine combos	GROUP BY cuisines
Q8	Highest rated cuisine combos	AVG(rate)
Q9	Niches	HAVING high rating and few restaurants
Q10	Cost vs rating	Categories + AVG cost and rating
Q11	Premium onboarding	Filter high cost + HAVING good rating
Q12	Busy but weak ratings	CTE + compare to overall average

Q13	Bundle features	CASE for both Yes vs other
Q14	Location success mix	% online, % booking, AVG rating
Q15	Top names per segment	ROW_NUMBER() OVER (PARTITION BY ... ORDER BY rate DESC)

Part 7 — Orders analytics (OQ1–OQ6)

ID	Question
OQ1	Revenue by payment method
OQ2	Discount yes/no vs average order value
OQ3	Revenue by weekday
OQ4	Top restaurants by revenue
OQ5	Revenue by month
OQ6	Payment × discount cross-tab revenue

Part 8 — Streamlit application (app.py)

Structure:

- **Sidebar:** Page selector — Dashboard, Q&A Analysis, Orders Analysis.
- **Dashboard:** User picks location, online order, book table, minimum rating → **parameterized SQL** → `st.dataframe`.
- **Q&A / Orders:** User picks a predefined question → button runs one SQL → table.

Why parameterized SQL:

Placeholders (?) and a parameter list avoid string injection and are good practice.

No charts:

Matches the project brief: tabular outputs only.

Part 9 — How to run (for you and your mentor)

```
```bash
pip install -r requirements.txt
```

## Copy Uber\_Eats\_data.csv and orders.json into project folder

```
python create_db.py
streamlit run app.py
```
```

Part 10 — Conclusion and business suggestions (for submission)

Conclusion (examples):

- Summarize 2–3 findings from **your** tables (e.g. mid segment vs rating, strong areas from Q1/Q11, order revenue by payment).
- Mention limitations: cuisine as combined string, sample orders if JSON omitted.

Business suggestions (examples):

- Prioritize onboarding in high-rating, premium-friendly locations.
 - Avoid oversaturated pockets unless you have a differentiation plan.
 - Use order and discount insights to tune promotions.
-

Part 11 — Live evaluation flow (about 30 minutes)

1. Self introduction (~3 min).
 2. Problem statement + walkthrough (Dashboard → one Q&A → one Order query) (~12 min).
 3. **Be ready:** Python/Pandas, SQL (GROUP BY, CASE, HAVING, CTE, window), Streamlit, SQLite, ETL, one statistical test.
 4. Mock questions (~10 min).
 5. Feedback (~5 min).
-

Part 12 — GitHub submission

Submit the **repository URL** in ZEN. The repo should contain **source code**, **README**, and this **guide** (and PDF if generated). Build `ubereats.db` locally after cloning; add CSV/JSON locally as required.

End of guide.